

Noise Filtering Techniques for a Vibration-Based Artificial Robotic Antenna

Contents

Abstract	2
1 Introduction	3
1.1 The Sensing Problem	3
1.2 Why Filtering Must Precede Ego-Motion Cancellation	4
1.3 Candidate Filtering Techniques	4
2 Low-Pass Filter (LPF)	5
2.1 Introduction	5
2.2 Principle of Operation	5
2.3 Why It Removes High-Frequency Noise	5
2.4 Block Diagram	5
2.5 Signal Flow Diagram	5
2.6 Frequency Response	6
2.7 Mathematical Derivation	6
2.7.1 Continuous-Time Equation	6
2.7.2 Discrete Implementation	7
2.7.3 Difference Equation and Variable Definitions	7
2.8 Cutoff Frequency Explanation	7
2.9 Selection of Cutoff Frequency	7
2.10 Computational Complexity	8
2.11 Memory Requirement	8
2.12 Advantages	8
2.13 Disadvantages	8
2.14 Embedded Implementation	8
2.15 Application in MPU6050	9
2.16 Suitability for Vibration-Based Tactile Sensing	9
3 Kalman Filter	10
3.1 Introduction	10
3.2 State Estimation Concept	10
3.3 Prediction Step	10
3.4 Correction Step	10
3.5 Covariance Estimation and Kalman Gain	10
3.6 State-Space Representation	11
3.7 Block Diagram	11
3.8 Flowchart: Prediction–Correction Loop	12
3.9 Variable Definitions	12
3.10 Process Noise, Measurement Covariance, and Uncertainty Weighting	13
3.11 Sensor Fusion Concept	13
3.12 Application to MPU6050 Accelerometer and Gyroscope Data	13

3.13	Computational Requirements: Arduino, ESP32, and Raspberry Pi	13
3.14	Advantages	14
3.15	Limitations	14
4	Madgwick Filter	15
4.1	Introduction	15
4.2	Quaternion Representation	15
4.3	Orientation Estimation via Gyroscope Integration	15
4.4	Accelerometer Correction via Gradient Descent	15
4.5	Sensor Fusion and Final Quaternion Update	16
4.6	Explanation of Key Quantities	16
4.7	Block Diagram	17
4.8	Flowchart	17
4.9	Why Madgwick Is Computationally Cheaper Than Kalman	18
4.10	Implementation Using MPU6050 Without Magnetometer	18
4.11	Computational Complexity	18
4.12	Advantages	18
4.13	Disadvantages	19
5	Comparison of Filtering Techniques	20
6	Application to This Project	21
6.1	Project Summary	21
6.2	Expected Performance of the Low-Pass Filter	21
6.3	Expected Performance of the Kalman Filter	21
6.4	Expected Performance of the Madgwick Filter	21
6.5	Filter Position Relative to the ESN Stage	22
7	Final Filter Selection	23
7.1	Real-Time Performance	23
7.2	Memory Usage	23
7.3	Computational Cost	23
7.4	Noise Suppression	23
7.5	Suitability for Arduino	23
7.6	Suitability for ESP32	23
7.7	Future Scalability	23
7.8	Engineering Recommendation	24
8	Conclusion	25
	References	26

Abstract

This report investigates noise filtering techniques suitable for a vibration-based artificial robotic antenna that uses an MPU6050 inertial measurement unit (IMU) mounted at the tip of a flexible antenna structure to sense contact-induced vibrations during active tactile exploration. Because the antenna is attached to a mobile robot driven by two servo motors, the raw accelerometer signal is corrupted by self-induced vibration arising from the robot's own motion (ego-motion), in addition to the vibrations of interest that arise from external contact. Left unfiltered, this self-generated noise significantly reduces the accuracy of any downstream machine-learning-based ego-motion cancellation stage, such as an Echo State Network (ESN). Three candidate filtering approaches are analysed in depth: the Low-Pass Filter (LPF), the Kalman Filter, and the Madgwick Filter. For each method, the underlying mathematical principle, discrete-time implementation, computational cost, memory footprint, and suitability for embedded deployment on Arduino and ESP32-class microcontrollers are examined. A structured comparison is then presented, followed by an application-specific evaluation in the context of the antenna project, culminating in an engineering recommendation for the most appropriate filter to precede the ESN-based ego-motion cancellation stage.

1 Introduction

Insects such as crickets and spiders rely on filiform hairs and antennae to sense minute air currents and surface contacts, extracting rich tactile information from very small mechanical displacements. Inspired by this biological strategy, the present project develops an artificial robotic antenna: a flexible mechanical whisker with an MPU6050 inertial measurement unit (IMU) rigidly mounted at its tip. As the antenna deflects due to contact with an obstacle, the IMU's tri-axial accelerometer and gyroscope register the resulting vibration, which is subsequently processed to infer contact events, surface properties, and relative position of obstacles.

The robot carrying the antenna is driven by two servo motors that provide locomotion and active scanning motion. This is where the central engineering problem of this report arises. The MPU6050, being rigidly coupled to the robot chassis through the antenna base, does not only sense vibration caused by *external contact* with the environment; it also senses vibration generated by the *robot's own actuators*. This second component is referred to throughout this report as **ego-motion-induced vibration**, or simply **self-induced vibration**.

1.1 The Sensing Problem

At any instant, the raw signal $x(t)$ obtained from the MPU6050 can be modelled as a superposition of two components:

$$x(t) = s_{\text{ext}}(t) + s_{\text{ego}}(t) + n(t) \quad (1)$$

where $s_{\text{ext}}(t)$ is the vibration signal of interest caused by external contact, $s_{\text{ego}}(t)$ is the self-induced vibration caused by servo motion, and $n(t)$ is broadband sensor noise (quantisation noise, thermal noise, and I2C bus jitter). Because $s_{\text{ego}}(t)$ and $s_{\text{ext}}(t)$ are physically superimposed on the same rigid structure, they cannot be separated by placement of additional sensors alone; instead, signal processing must be used either to suppress $s_{\text{ego}}(t)$ directly or to condition the composite signal so that a downstream learning-based model, such as an Echo State Network (ESN), can learn to separate the two components.

Figure 1 illustrates the overall sensing pipeline, from the physical vibration source through to the ego-motion-cancelled tactile signal.

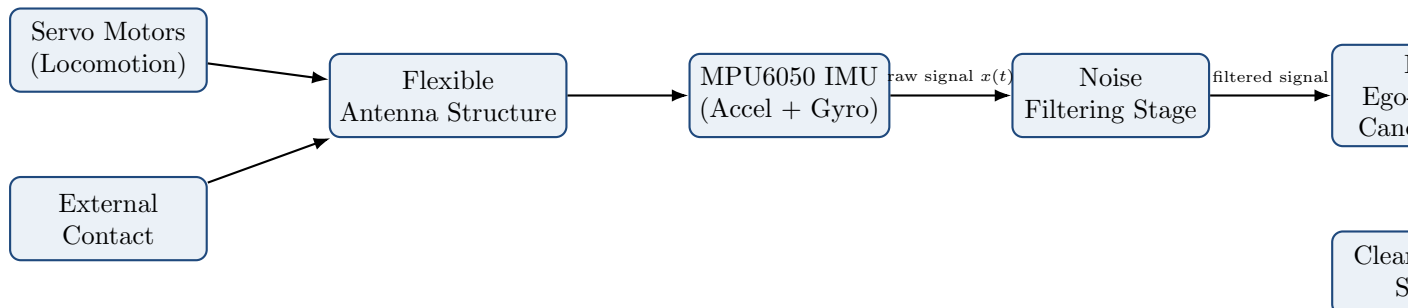


Figure 1: Overall signal processing pipeline: physical vibration sources, IMU sensing, the noise filtering stage addressed in this report, and downstream ego-motion cancellation.

1.2 Why Filtering Must Precede Ego-Motion Cancellation

The ESN-based ego-motion cancellation stage is a data-driven, recurrent model trained to predict and subtract the self-induced component $s_{\text{ego}}(t)$ from the composite signal. Its training and inference accuracy depend heavily on the signal-to-noise ratio of its input; a raw MPU6050 signal contaminated by high-frequency sensor noise and drift makes it substantially harder for the reservoir network to learn a stable mapping between servo commands and the corresponding vibration signature. Pre-filtering therefore serves two purposes: (i) it removes noise components that carry no useful information for either contact detection or ego-motion prediction, and (ii) it conditions the signal into a numerically well-behaved form (e.g., stable orientation estimates rather than raw, drifting angular data) that the ESN can use more effectively.

1.3 Candidate Filtering Techniques

Three filtering techniques, widely used in embedded IMU applications, are evaluated as candidates for this pre-processing stage:

- (i) **Low-Pass Filter (LPF):** A simple, first-order recursive filter that attenuates high-frequency components above a chosen cutoff frequency. It is computationally minimal and well suited to noisy accelerometer data where the vibration of interest lies below a known frequency band.
- (ii) **Kalman Filter:** An optimal recursive state estimator that fuses noisy measurements with a dynamic model of the system to produce a statistically optimal estimate of the true state (e.g., orientation or acceleration), while simultaneously quantifying the uncertainty of that estimate.
- (iii) **Madgwick Filter:** A computationally efficient orientation filter that fuses gyroscope and accelerometer data using a gradient-descent-based correction of a quaternion representation, without requiring the covariance bookkeeping of a Kalman filter.

These three techniques span a spectrum from very low computational cost (LPF) to high estimation accuracy at higher computational cost (Kalman), with the Madgwick filter occupying an intermediate position that is specifically optimised for orientation estimation on resource-constrained embedded platforms. The remainder of this report analyses each technique in detail, compares them systematically, and applies the analysis to the specific constraints of the robotic antenna project.

2 Low-Pass Filter (LPF)

2.1 Introduction

The Low-Pass Filter is the simplest and most computationally economical of the three candidate techniques. In the context of the robotic antenna, the LPF is used to attenuate the high-frequency components of the raw accelerometer signal, which are dominated by sensor quantisation noise and high-frequency mechanical resonances of the antenna shaft, while preserving the lower-frequency vibration content that carries information about contact events and gross servo motion.

2.2 Principle of Operation

A low-pass filter permits signal components below a cutoff frequency f_c to pass through largely unattenuated, while progressively attenuating components above f_c . This is desirable in the antenna application because the sensor noise floor and micro-vibrations of the servo gear train occupy a broadband, relatively high-frequency spectrum, whereas the mechanically significant bending modes of the flexible antenna and the majority of the useful contact-vibration energy lie within a lower and more predictable frequency band.

2.3 Why It Removes High-Frequency Noise

Physically, the LPF acts as a moving average with exponentially decaying weights on past samples. High-frequency noise consists of rapid sample-to-sample fluctuations; because such fluctuations tend to average toward zero over a short window, the exponential averaging operation of the LPF suppresses them, while slower-varying (low-frequency) trends persist across many samples and therefore survive the averaging process largely intact.

2.4 Block Diagram

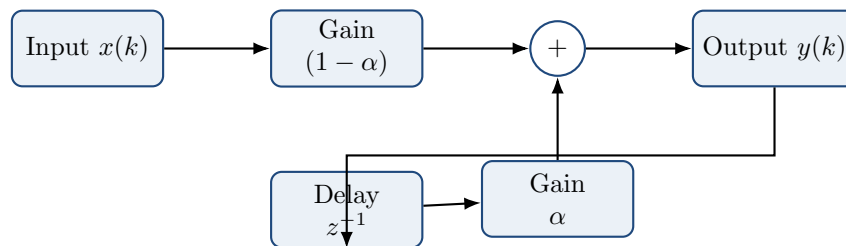


Figure 2: Block diagram of the first-order recursive low-pass filter, showing the feedback path that implements exponential averaging.

2.5 Signal Flow Diagram

Figure 3 shows the equivalent signal flow: the current sample is weighted and combined with a feedback term derived from the previous output, producing a smoothed estimate at every discrete time step k .

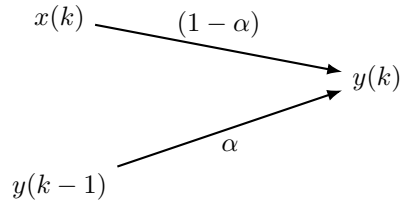


Figure 3: Signal flow representation of the exponential moving average.

2.6 Frequency Response

The frequency response magnitude of the first-order recursive LPF, for a normalised frequency $\omega = 2\pi f/f_s$ (where f_s is the sampling frequency), is given by

$$|H(e^{j\omega})| = \frac{1 - \alpha}{\sqrt{1 - 2\alpha \cos \omega + \alpha^2}}. \quad (2)$$

Figure 4 plots this magnitude response for three representative values of α , illustrating how a larger α (closer to 1) produces a lower cutoff frequency and stronger attenuation of high-frequency content, at the cost of increased phase lag.

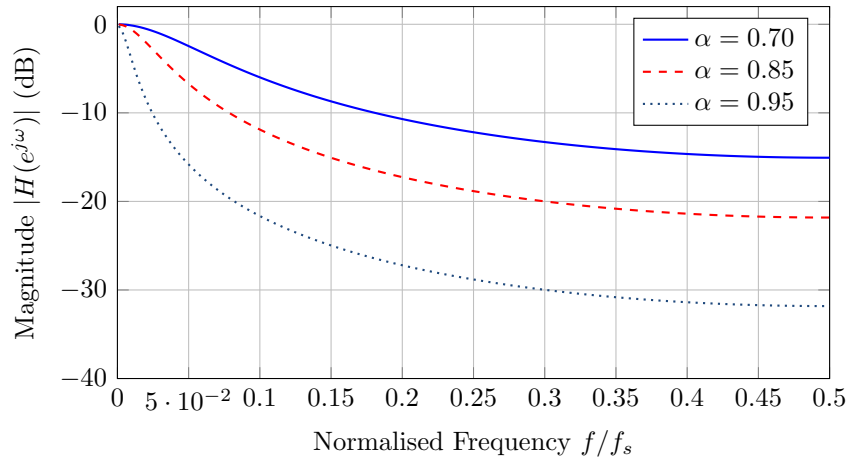


Figure 4: Magnitude frequency response of the first-order LPF for three values of the smoothing coefficient α . Larger α pushes the cutoff frequency lower and increases high-frequency attenuation.

2.7 Mathematical Derivation

2.7.1 Continuous-Time Equation

The analogue first-order low-pass filter is described by the ordinary differential equation

$$\tau \frac{dy(t)}{dt} + y(t) = x(t), \quad (3)$$

where $\tau = 1/(2\pi f_c)$ is the filter time constant and f_c is the cutoff frequency. Physically, this equation states that the rate of change of the output is proportional to the difference between the input and the current output; the output therefore "chases" the input with a lag set by τ .

2.7.2 Discrete Implementation

Discretising Equation (3) using a first-order backward-difference (Euler) approximation with sampling period $T_s = 1/f_s$ yields the standard discrete recursive form

$$y(k) = \alpha y(k-1) + (1-\alpha)x(k), \quad (4)$$

where the smoothing coefficient α is related to the sampling period and time constant by

$$\alpha = \frac{\tau}{\tau + T_s} = e^{-T_s/\tau}. \quad (5)$$

2.7.3 Difference Equation and Variable Definitions

The difference equation implemented in firmware is

$$y(k) = \alpha y(k-1) + (1-\alpha)x(k) \quad (6)$$

where:

- $x(k)$ – the raw accelerometer or gyroscope sample at discrete time step k ;
- $y(k)$ – the filtered output at time step k ;
- $y(k-1)$ – the filtered output computed at the previous time step, held in a single memory register;
- $\alpha \in [0, 1)$ – the smoothing coefficient that governs the trade-off between noise suppression and responsiveness; and
- k – the discrete sample index, incremented at each fixed sampling interval T_s .

2.8 Cutoff Frequency Explanation

The cutoff frequency f_c is the frequency at which the filter's power attenuation reaches -3 dB (approximately 70.7% of the input amplitude). Frequencies below f_c pass through with little attenuation, while frequencies above f_c are progressively suppressed. Selecting f_c is therefore a direct trade-off: a low f_c produces strong noise rejection but introduces significant lag, which is undesirable for a system that must respond quickly to a fast contact event; a high f_c preserves fast dynamics but admits more high-frequency noise.

2.9 Selection of Cutoff Frequency

For this project, the cutoff frequency should be chosen with reference to two physical bandwidths: (i) the fastest contact transient the antenna is expected to detect, and (ii) the dominant mechanical resonance frequency of the servo gear train. Typically, hobby-servo-induced vibration and MPU6050 quantisation noise occupy the band above approximately 40–60 Hz, while the antenna's flexural response to contact events is expected to lie below approximately 20–30 Hz. A cutoff frequency in the range of $f_c \approx 15$ –25 Hz, corresponding to $\alpha \approx 0.85$ –0.93 at a typical

sampling rate of $f_s = 200$ Hz, provides a reasonable starting point, to be refined empirically using recorded vibration data from both idle-servo and active-contact conditions.

2.10 Computational Complexity

Each output sample requires exactly one multiplication of the previous output by α , one multiplication of the current input by $(1 - \alpha)$, and one addition, i.e., $O(1)$ arithmetic operations per sample, independent of any history length. This is the lowest computational complexity among the three candidate filters.

2.11 Memory Requirement

The filter requires storage for only a single scalar state variable, $y(k - 1)$, per filtered channel. For a 6-axis IMU (three accelerometer axes and three gyroscope axes), this amounts to six floating-point (or fixed-point) registers, i.e., a memory footprint on the order of tens of bytes.

2.12 Advantages

- Extremely low computational cost, executable in microseconds even on an 8-bit microcontroller.
- Minimal memory footprint (a single state variable per channel).
- Simple to tune (a single parameter, α or equivalently f_c).
- Numerically stable for all $\alpha \in [0, 1)$; no risk of divergence.
- No dependency on an accurate dynamic model of the system.

2.13 Disadvantages

- Introduces phase lag that increases with α , delaying detection of fast contact transients.
- Cannot distinguish between a wanted low-frequency signal and unwanted low-frequency drift; it merely acts on frequency content, not on the true physical origin of the signal.
- Does not fuse gyroscope and accelerometer information, and does not provide an orientation estimate.
- Provides no explicit estimate of measurement uncertainty.
- A fixed α is a static compromise; it is not adaptive to changing noise conditions (e.g., when the servo is idle versus actively moving).

2.14 Embedded Implementation

The recursive LPF is trivially implemented within the IMU polling interrupt service routine or main sampling loop of an Arduino/ATmega328P or ESP32 firmware, requiring only a single line of code per axis:

```
y[i] = alpha * y_prev[i] + (1 - alpha) * x[i];
```

Fixed-point implementations (e.g., Q15 or Q16.16 formats) are straightforward and avoid the overhead of floating-point emulation on 8-bit microcontrollers such as the ATmega328P.

2.15 Application in MPU6050

When applied to the MPU6050's raw accelerometer and gyroscope outputs, the LPF suppresses high-frequency quantisation noise and mechanical ringing above the chosen cutoff, producing a smoother signal suitable for either direct thresholding (contact detection) or as a lower-noise input to the ESN-based ego-motion cancellation stage. Notably, the MPU6050 also contains an internal hardware Digital Low-Pass Filter (DLPF) with several selectable bandwidths (5 Hz to 260 Hz), which may be used in conjunction with, or instead of, a software LPF stage.

2.16 Suitability for Vibration-Based Tactile Sensing

The LPF is well suited as a first-stage, low-cost noise-reduction step, particularly for removing broadband sensor noise before any higher-level processing. However, because it treats the self-induced servo vibration and the external contact vibration purely on the basis of frequency content – and both signals can share overlapping frequency bands – the LPF alone cannot reliably separate ego-motion from contact-induced vibration. It is therefore best regarded as a complementary pre-processing stage rather than a complete solution.

3 Kalman Filter

3.1 Introduction

The Kalman Filter is a recursive Bayesian estimator that produces the statistically optimal estimate (in the minimum mean-squared-error sense, for linear-Gaussian systems) of the internal state of a dynamic system, by fusing a mathematical model of the system's expected evolution with noisy sensor measurements. In the context of the robotic antenna, the Kalman Filter can be used to estimate quantities such as the antenna tip's true orientation or angular velocity from the MPU6050's noisy accelerometer and gyroscope readings, while explicitly accounting for the relative trustworthiness of the model prediction versus the sensor measurement.

3.2 State Estimation Concept

Unlike the LPF, which only smooths an incoming signal, the Kalman Filter maintains an internal belief about the *true state* of the system – e.g., orientation angles and angular rates – represented as a state vector $\hat{x}_k$ together with a covariance matrix P_k describing the estimator's confidence in that state. At every time step, this belief is updated in two stages: a *prediction* step that propagates the belief forward in time using a dynamic model, and a *correction* (or update) step that adjusts the predicted belief using the newly arrived sensor measurement.

3.3 Prediction Step

Given the state estimate and covariance from the previous time step, the prediction step projects them forward using the system's linear dynamic model:

$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_k \quad (7)$$

$$P_k^- = AP_{k-1}A^T + Q \quad (8)$$

3.4 Correction Step

Once a new measurement z_k arrives, the Kalman gain is computed and used to correct the predicted state and covariance:

$$K_k = P_k^- H^T (HP_k^- H^T + R)^{-1} \quad (9)$$

$$\hat{x}_k = \hat{x}_k^- + K_k (z_k - H\hat{x}_k^-) \quad (10)$$

$$P_k = (I - K_k H)P_k^- \quad (11)$$

3.5 Covariance Estimation and Kalman Gain

The Kalman Gain K_k determines how much weight is given to the new measurement relative to the model's prediction. When the measurement noise covariance R is small relative to the predicted state covariance P_k^- , K_k approaches a value that weights the measurement heavily; conversely, when R is large (a noisy sensor) relative to P_k^- , K_k shrinks and the filter relies more on the model's prediction. This automatic, statistically-grounded balancing is the central

strength of the Kalman Filter and is what distinguishes it from a fixed-coefficient filter such as the LPF.

3.6 State-Space Representation

For the antenna's IMU-based orientation tracking, a simple discrete-time state-space model can be formed using the pitch and roll angles and their respective angular rate biases as the state vector:

$$\hat{x}_k = \begin{bmatrix} \theta_k \\ b_k \end{bmatrix}, \quad A = \begin{bmatrix} 1 & -T_s \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} T_s \\ 0 \end{bmatrix} \quad (12)$$

where θ_k is the estimated tilt angle, b_k is the estimated gyroscope bias, T_s is the sampling period, and the control input u_k is the gyroscope's measured angular rate. The measurement model uses the accelerometer-derived tilt angle as the observation, with $H = \begin{bmatrix} 1 & 0 \end{bmatrix}$.

3.7 Block Diagram

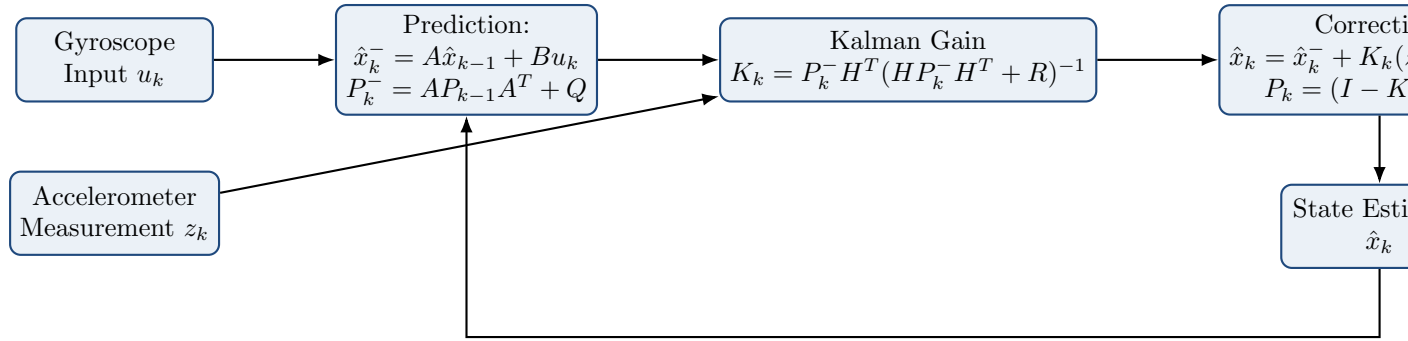


Figure 5: Block diagram of the discrete Kalman filter, showing the prediction stage, Kalman gain computation, and correction stage, with feedback of the updated state to the next prediction cycle.

3.8 Flowchart: Prediction–Correction Loop

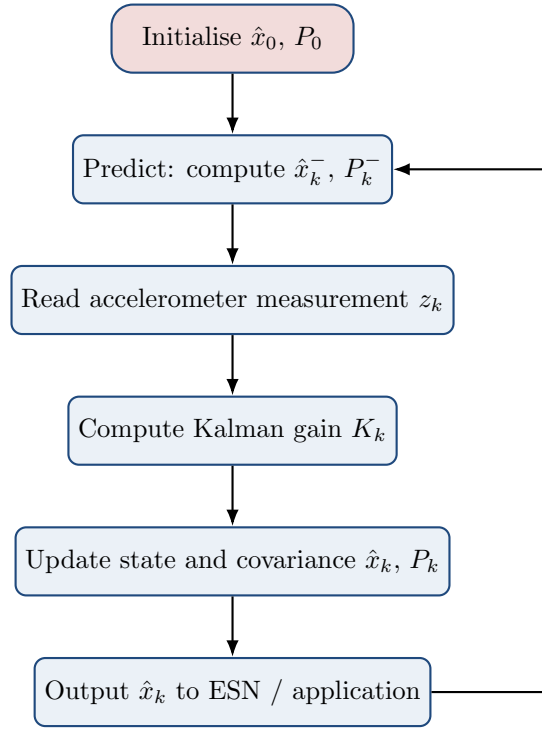


Figure 6: Flowchart of the recursive Kalman filter predict-correct loop as executed at every sample interval $k \rightarrow k + 1$.

3.9 Variable Definitions

- $\hat{x}_k$ – the *a posteriori* state estimate at time k , after incorporating the measurement;
- $\hat{x}_k^-$ – the *a priori* state estimate at time k , predicted before the measurement is incorporated;
- A – the state transition matrix describing how the state evolves in the absence of noise;
- B – the control input matrix, mapping the control input u_k (e.g., gyroscope rate) into the state;
- P_k, P_k^- – the *a posteriori* and *a priori* error covariance matrices, representing the estimator’s uncertainty about the state;
- Q – the process noise covariance matrix, representing uncertainty in the dynamic model itself;
- R – the measurement noise covariance matrix, representing the trustworthiness of the sensor;
- H – the observation matrix mapping the state space into the measurement space;
- z_k – the actual sensor measurement at time k ;

- K_k – the Kalman gain, a matrix that determines the relative weighting of prediction versus measurement; and
- I – the identity matrix.

3.10 Process Noise, Measurement Covariance, and Uncertainty Weighting

The process noise covariance Q reflects how much the true state is expected to deviate from the model’s noise-free prediction – for instance, unmodelled gyroscope bias drift or unexpected antenna dynamics. A larger Q tells the filter to trust its own model prediction less. The measurement noise covariance R reflects the expected noise level of the accelerometer-derived measurement; a larger R tells the filter to trust the incoming measurement less. The Kalman Gain in Equation (9) automatically balances these two competing sources of uncertainty at every time step, which is precisely what gives the Kalman Filter its adaptive, statistically optimal character, in contrast to the LPF’s fixed weighting.

3.11 Sensor Fusion Concept

The Kalman Filter is a natural framework for sensor fusion: the gyroscope, which is accurate over short time intervals but drifts over long ones, provides the prediction (via the $A\hat{x}_{k-1} + Bu_k$ term), while the accelerometer, which is noisy at high frequency (due to vibration) but bias-free over long intervals (since it senses the true gravity vector on average), provides the correction. This complementary combination – fast, drifting gyroscope predictions corrected by slow, stable accelerometer measurements – is the essence of Kalman-based IMU sensor fusion, and is the mechanism used in Section 4 by the Madgwick Filter as well.

3.12 Application to MPU6050 Accelerometer and Gyroscope Data

For the antenna project, a Kalman Filter can be configured with the state vector representing tilt angle and gyroscope bias (as in the state-space model above), using the gyroscope’s integrated angular rate as the prediction input and the accelerometer-derived tilt angle (computed via arctan of the gravity components) as the correction measurement. The resulting filtered orientation estimate is far more stable than either sensor alone, and importantly, provides a running estimate of gyroscope bias, which is valuable for removing slow orientation drift accumulated during extended active-scanning sequences.

3.13 Computational Requirements: Arduino, ESP32, and Raspberry Pi

- **Arduino (ATmega328P, 16 MHz, no FPU):** A minimal 2-state Kalman filter (e.g., the well-known “Kalman1D” single-axis implementation) is feasible in real time at typical IMU sampling rates (50–200 Hz), but a full 4-state or higher-order multi-axis Kalman filter with matrix inversion may strain the 8-bit core, particularly if implemented in floating point without hardware multiply-accumulate support.
- **ESP32 (Xtensa dual-core, 240 MHz, hardware floating point):** Comfortably executes multi-axis Kalman filters, including full quaternion-based Extended Kalman Filters

(EKF), with ample timing margin at typical IMU sampling rates.

- **Raspberry Pi (ARM Cortex-A, full Linux, floating point, multi-core):** Executes even large-scale, multi-sensor Kalman or Extended Kalman filters with negligible computational burden, and is suitable for higher-order models incorporating additional sensors.

3.14 Advantages

- Statistically optimal (minimum mean-squared error) for linear systems with Gaussian noise.
- Explicitly models and propagates uncertainty, enabling principled sensor fusion.
- Adaptively balances trust between model prediction and sensor measurement via the Kalman gain.
- Naturally estimates auxiliary states such as gyroscope bias, which a simple LPF cannot.
- Well established, with decades of aerospace, robotics, and navigation heritage.

3.15 Limitations

- Requires an accurate linear (or linearised) dynamic model; performance degrades if the model is a poor match to the true system.
- Computationally heavier than the LPF, requiring matrix multiplications and, in general form, a matrix inversion at every step.
- Requires careful tuning of Q and R , which is often done empirically and can be non-trivial.
- The basic (linear) Kalman Filter is not directly applicable to the nonlinear quaternion kinematics used for full 3D orientation without extension to an Extended Kalman Filter (EKF) or Unscented Kalman Filter (UKF), increasing implementation complexity.

4 Madgwick Filter

4.1 Introduction

The Madgwick Filter is an orientation estimation algorithm developed specifically for low-power, embedded IMU and IMU/magnetometer (AHRS) applications. It estimates orientation as a quaternion by integrating gyroscope data over time and correcting the resulting drift using a gradient-descent step derived from accelerometer (and, optionally, magnetometer) measurements. Its principal appeal for the antenna project is that it achieves accuracy competitive with a Kalman-based approach while requiring substantially less computation and no matrix inversion.

4.2 Quaternion Representation

Orientation is represented using a unit quaternion

$$q = q_0 + q_1i + q_2j + q_3k, \quad \|q\| = 1, \quad (13)$$

where q_0 is the scalar part and q_1, q_2, q_3 form the vector part. Quaternions avoid the gimbal-lock singularities of Euler angles and are computationally cheaper to update than full rotation matrices, making them well suited to embedded orientation tracking.

4.3 Orientation Estimation via Gyroscope Integration

The rate of change of orientation is related to the measured angular velocity $\omega = [0, \omega_x, \omega_y, \omega_z]$ (expressed as a pure quaternion) by the kinematic relationship

$$\dot{q}_{\omega,k} = \frac{1}{2} q_{k-1} \otimes \omega_k, \quad (14)$$

where $\otimes$ denotes quaternion multiplication. Integrating this rate over one sample interval T_s gives a gyroscope-only orientation estimate:

$$q_{\omega,k} = q_{k-1} + \dot{q}_{\omega,k} T_s. \quad (15)$$

As with any pure integration, Equation (15) accumulates drift over time due to gyroscope bias and noise, motivating the accelerometer-based correction described next.

4.4 Accelerometer Correction via Gradient Descent

The Madgwick algorithm corrects the gyroscope-integrated estimate by defining an objective function $f(q, a)$ that measures the error between the direction of gravity predicted by the current orientation estimate and the direction of gravity actually measured by the accelerometer $a = [a_x, a_y, a_z]$:

$$f(q, a) = R(q)^T \hat{g} - \hat{a}, \quad (16)$$

where $\hat{g} = [0, 0, 1]^T$ is the reference gravity direction in the earth frame and $\hat{a}$ is the normalised accelerometer measurement. The gradient of this objective function with respect to the quaternion components is

$$\nabla f = J^T(q) f(q, a), \quad (17)$$

where $J(q)$ is the Jacobian of f with respect to q . A single step of gradient descent then nudges the quaternion estimate in the direction that reduces this error:

$$q_{\nabla, k} = q_{k-1} - \beta \frac{\nabla f}{\|\nabla f\|}, \quad (18)$$

where β is a tunable step-size parameter, discussed below.

4.5 Sensor Fusion and Final Quaternion Update

The gyroscope-integrated estimate and the gradient-descent correction are fused by a weighted combination:

$$\dot{q}_k = \dot{q}_{\omega, k} - \beta \frac{\nabla f}{\|\nabla f\|}, \quad (19)$$

$$q_k = q_{k-1} + \dot{q}_k T_s, \quad (20)$$

followed by re-normalisation of the resulting quaternion to maintain the unit-norm constraint:

$$q_k \leftarrow \frac{q_k}{\|q_k\|}. \quad (21)$$

4.6 Explanation of Key Quantities

- **Quaternion (q):** A four-component representation of 3D orientation that is free of gimbal lock and computationally efficient to update and compose.
- **Objective function (f):** A measure of how well the orientation estimate's predicted gravity direction matches the accelerometer's measured gravity direction; it is zero when the estimate is fully consistent with the accelerometer reading.
- **Gradient (∇f):** The direction in quaternion space in which the objective function increases most rapidly; moving *against* this direction reduces the orientation error.
- **Beta parameter (β):** A gain that determines how strongly the accelerometer-based correction is trusted relative to the gyroscope integration. A larger β produces faster convergence and stronger noise rejection from accelerometer disturbances, but slower response to fast, genuine orientation changes; a smaller β trusts the gyroscope more, tracking fast motion accurately, but is more susceptible to long-term drift.

4.7 Block Diagram

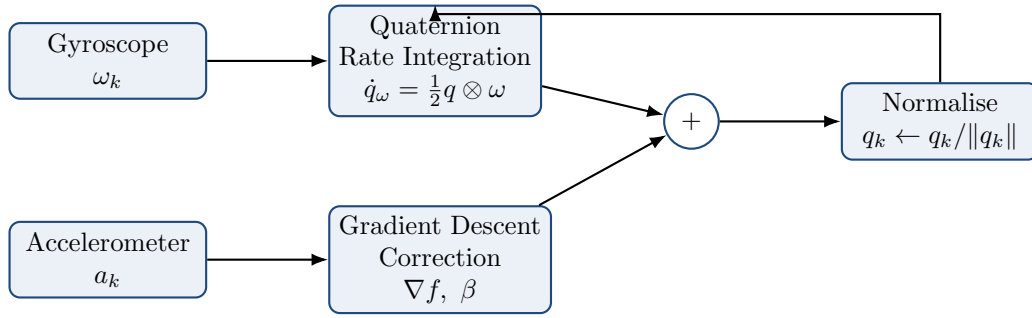


Figure 7: Block diagram of the Madgwick orientation filter: gyroscope rate integration is fused with an accelerometer-derived gradient-descent correction, and the combined result is renormalised at each step.

4.8 Flowchart

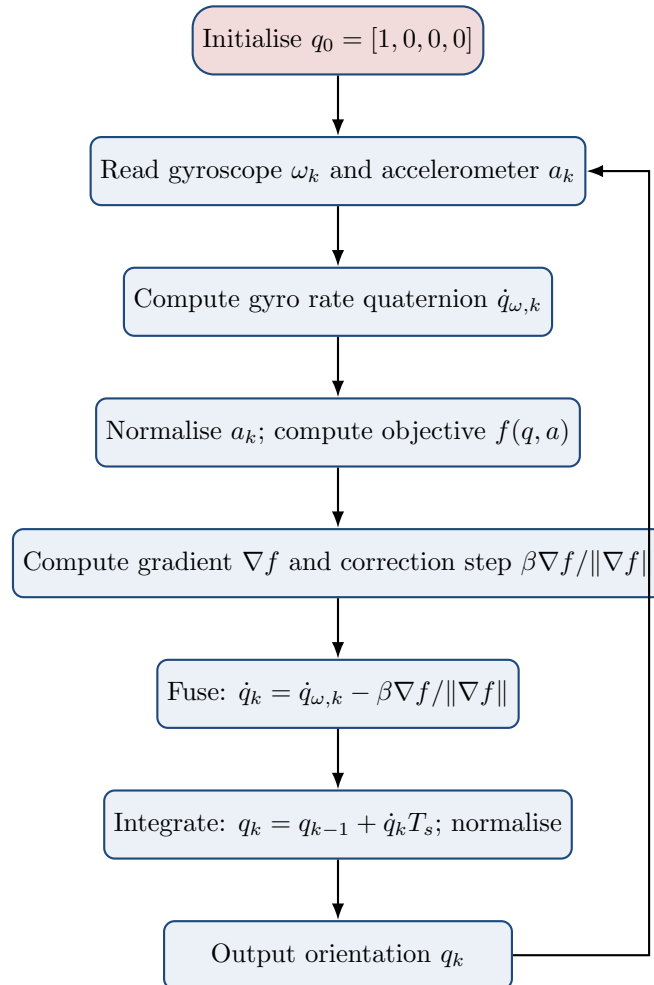


Figure 8: Flowchart of the Madgwick gradient-descent orientation filter update cycle.

4.9 Why Madgwick Is Computationally Cheaper Than Kalman

The Madgwick filter replaces the Kalman filter’s matrix operations – state transition matrix multiplication, covariance propagation, and, critically, matrix inversion in the gain computation – with a fixed sequence of quaternion multiplications, a small number of trigonometric-free algebraic gradient terms, and a single normalisation step. Because no matrix inversion is required and no covariance matrix is propagated, both the arithmetic operation count and the memory footprint are substantially lower than an equivalent Kalman-based orientation filter, while the analytically derived gradient expressions (as given in Madgwick’s original formulation) can be implemented as a short, fixed sequence of multiply-accumulate operations well suited to an 8-bit or low-power 32-bit microcontroller.

4.10 Implementation Using MPU6050 Without Magnetometer

The full Madgwick AHRS algorithm was originally formulated to fuse gyroscope, accelerometer, and magnetometer data for full 3D heading, pitch, and roll estimation. Because the MPU6050 (unlike the MPU9250) does not include an on-board magnetometer, the antenna project uses the reduced IMU-only variant of the algorithm (often referred to as the “Madgwick IMU update”), which fuses only gyroscope and accelerometer data. This variant estimates pitch and roll reliably, since these are observable from the gravity vector alone, but cannot observe absolute yaw (heading), which would require a magnetometer or an alternative yaw reference. For the antenna’s vibration and tilt sensing role, where absolute heading is not required, this is an acceptable and appropriate simplification.

4.11 Computational Complexity

Each update requires on the order of a few dozen floating-point multiply-accumulate operations (quaternion products, the gradient computation, and normalisation), with no matrix inversion and no covariance matrix of size greater than 4×4 elements to maintain (and in the IMU-only variant, no covariance matrix at all). This places the Madgwick filter’s computational cost between that of the LPF and a full Kalman/EKF implementation, but considerably closer to the LPF in practice, which is the primary reason it is favoured for real-time orientation estimation on 8-bit and low-power 32-bit embedded platforms.

4.12 Advantages

- Computationally efficient: no matrix inversion, suitable for real-time execution on modest microcontrollers.
- Provides a full orientation (quaternion) estimate, unlike the scalar LPF.
- Single tunable gain (β) offers an intuitive trade-off between responsiveness and noise rejection.
- Numerically robust due to quaternion normalisation at every step.
- Widely validated in open-source AHRS/IMU libraries and embedded robotics projects.

4.13 Disadvantages

- Does not provide an explicit statistical measure of estimation uncertainty (no covariance output), unlike the Kalman filter.
- Performance depends on empirical tuning of β , without the principled noise-covariance interpretation of Q and R in the Kalman filter.
- Without a magnetometer, yaw (heading) is not observable and will drift freely.
- Assumes the accelerometer measurement predominantly reflects gravity; this assumption is degraded during large linear accelerations (e.g., a sharp contact impact), which is a specific concern for the antenna application.

5 Comparison of Filtering Techniques

Table 1 summarises the three candidate filtering techniques across the criteria most relevant to embedded, vibration-based tactile sensing.

Criterion	Low-Pass Filter	Kalman Filter	Madgwick Filter
Working principle	Exponential moving average in frequency domain	Recursive Bayesian state estimation with model-measurement fusion	Quaternion integration corrected by gradient-descent accelerometer fusion
Noise removal capability	Good for broadband high-frequency noise	Excellent; statistically optimal given accurate model	Good; effective for accelerometer noise and short-term disturbance
Orientation estimation	Not provided	Full state estimate (can include orientation)	Full orientation (quaternion) natively
Prediction capability	None (purely reactive)	Explicit model-based prediction step	Implicit, via gyroscope integration
Sensor fusion	None	Full statistical fusion (gyro + accel, extensible)	Gyro + accel fusion (no magnetometer here)
Computational complexity	$O(1)$, one multiply-add per sample	Moderate-high; matrix multiply and inversion each step	Low-moderate; quaternion algebra, no matrix inversion
Memory requirement	Minimal (1 scalar per channel)	Higher (state vector + covariance matrix)	Low (4-element quaternion, no covariance)
Execution speed	Fastest	Slowest of the three	Fast; close to LPF in practice
Embedded suitability	Excellent on any platform	Best on 32-bit MCUs with FPU	Excellent on 8-bit and 32-bit MCUs
Arduino (ATmega328P) suitability	Excellent	Feasible only for reduced-order models	Good, well within real-time budget
ESP32 suitability	Excellent	Excellent	Excellent
Real-time capability	Yes, trivially	Yes, with careful implementation	Yes
Ease of implementation	Very simple (one line of code)	Moderate-complex (matrix algebra, tuning Q, R)	Moderate (well-documented reference implementations)
Advantages	Simplicity, speed, low memory	Statistically optimal, uncertainty-aware, bias estimation	Efficient orientation estimate, single intuitive gain
Disadvantages	No orientation output, static trade-off, phase lag	Higher cost, requires accurate model, tuning burden	No uncertainty output, yaw not observable without magnetometer
Typical applications	Sensor pre-conditioning, audio, basic smoothing	Navigation, robotics localisation, sensor fusion, tracking	Wearables, drones, AHRS, low-power orientation tracking

Table 1: Comprehensive comparison of the Low-Pass, Kalman, and Madgwick filters across criteria relevant to embedded vibration-based tactile sensing.

6 Application to This Project

6.1 Project Summary

To ground the comparison of Section 5 in the specific constraints of this project, the key characteristics of the antenna system are restated here:

- The robot moves using two servo motors, which are the dominant source of self-induced (ego-motion) vibration.
- An MPU6050 IMU is rigidly mounted at the tip of a flexible antenna, sensing both self-induced and contact-induced vibration.
- The robot performs active tactile sensing, sweeping the antenna to probe its surroundings.
- Future work will employ an Echo State Network (ESN) to learn and subtract the servo-induced ego-motion component from the composite IMU signal.

6.2 Expected Performance of the Low-Pass Filter

Because servo-induced vibration and contact-induced vibration are likely to share overlapping frequency content (both are mechanical vibrations transmitted through the same flexible structure), a simple LPF cannot cleanly separate the two. It can, however, usefully remove very-high-frequency sensor noise and antenna resonance ringing that carries no information relevant to either ego-motion or contact, providing a cleaner input signal to whichever downstream stage handles the actual separation task.

6.3 Expected Performance of the Kalman Filter

A Kalman Filter, particularly one incorporating a dynamic model of the antenna’s mechanical response (e.g., a damped oscillator model of the flexible shaft) and of the servo motion profile, could in principle provide the best statistical estimate of the antenna’s true orientation and vibration state, including a meaningful estimate of measurement uncertainty at each point in time. However, constructing an accurate linear dynamic model that captures both the antenna’s flexural dynamics and the highly repeatable but nonlinear servo motion profile is non-trivial, and the added computational and tuning burden may not be justified if the downstream ESN is expected to learn the ego-motion relationship directly from data.

6.4 Expected Performance of the Madgwick Filter

The Madgwick Filter is well matched to this project’s core requirement: a computationally efficient, drift-corrected estimate of the antenna tip’s orientation (pitch and roll), obtained without requiring an explicit dynamic model of the antenna or the servo motion. Because it does not depend on an accurate system model, it is robust to the fact that the antenna’s precise mechanical parameters (stiffness, damping) may not be well characterised. Its single tuning gain (β) is also more straightforward to tune empirically than the Kalman Filter’s process and measurement covariance matrices.

6.5 Filter Position Relative to the ESN Stage

A key architectural question is whether noise filtering should occur *before*, *after*, or *together with* the ESN-based ego-motion cancellation stage.

- **Before the ESN (recommended):** Applying the LPF and/or Madgwick filter before the ESN conditions the raw signal into a lower-noise, orientation-stable input, which is expected to improve the ESN’s ability to learn a consistent mapping between servo commands and the resulting ego-motion signature, since it is not forced to simultaneously learn to reject broadband sensor noise.
- **After the ESN:** Filtering after the ESN would only smooth the already ego-motion-cancelled output, and would not help the ESN’s internal learning process handle noisy inputs; it also risks re-introducing phase lag into a signal that may already be intended for time-critical contact detection.
- **Together with the ESN:** It is possible, in principle, to let the ESN itself learn to perform noise rejection implicitly, foregoing an explicit filtering stage entirely. This is generally undesirable here, since it increases the burden on the ESN’s limited reservoir dynamics and makes the overall pipeline harder to validate, tune, and debug in isolation.

Figure 9 illustrates the recommended architecture, in which the Madgwick filter (optionally preceded by a light LPF pre-stage on the raw accelerometer channel) is placed before the ESN.



Figure 9: Recommended signal processing pipeline placing the Madgwick filter (with an optional light LPF pre-stage) before the ESN-based ego-motion cancellation network.

7 Final Filter Selection

7.1 Real-Time Performance

All three filters are capable of real-time operation at typical IMU sampling rates (50–200 Hz). However, the Madgwick filter offers a better balance of update rate headroom versus estimation quality than the LPF (which provides no orientation estimate at all) and the Kalman filter (whose matrix operations consume a larger fraction of the available time budget on 8-bit hardware).

7.2 Memory Usage

The Madgwick filter’s memory requirement (a 4-element quaternion state, with no covariance matrix) is only marginally larger than the LPF’s single-scalar state and substantially smaller than the Kalman filter’s state-plus-covariance-matrix requirement, which scales with the square of the state dimension.

7.3 Computational Cost

The Madgwick filter requires no matrix inversion, unlike the Kalman filter, and its fixed sequence of quaternion and gradient operations executes comfortably within the timing budget of both the ATmega328P (Arduino) and ESP32 platforms used elsewhere in this project.

7.4 Noise Suppression

The Madgwick filter’s accelerometer-based correction, governed by the single gain β , provides effective rejection of short-term accelerometer noise and disturbance while preserving the fast dynamics captured by the gyroscope, achieving a favourable balance for tracking transient contact events.

7.5 Suitability for Arduino

Given the ATmega328P’s lack of a hardware floating-point unit and limited RAM, the Madgwick filter’s absence of matrix inversion makes it considerably more tractable than a full Kalman implementation, while still providing a genuine orientation estimate that the LPF cannot.

7.6 Suitability for ESP32

On the ESP32, all three filters, including a full EKF-based Kalman implementation, are computationally feasible. The Madgwick filter nonetheless remains attractive here for consistency with the Arduino-based prototype and for its lower tuning complexity relative to the Kalman filter’s covariance matrices.

7.7 Future Scalability

Should the project later require a magnetometer for absolute heading, or a more rigorous, uncertainty-aware fusion (e.g., to support probabilistic reasoning within the ESN training pipeline or a future sensor-fusion research contribution), the architecture can be extended to an Extended

Kalman Filter without discarding the Madgwick-based prototype, since both approaches operate on the same underlying quaternion/orientation representation.

7.8 Engineering Recommendation

Based on the above analysis, the **Madgwick Filter** is recommended as the primary orientation and vibration-conditioning stage for this project, applied to the MPU6050's raw gyroscope and accelerometer data prior to the ESN-based ego-motion cancellation stage, optionally supplemented by a light LPF pre-stage on the raw accelerometer channel to remove very-high-frequency sensor noise before it enters the Madgwick update. The **Kalman Filter** is retained as a documented alternative, to be revisited if the project scales to a more computationally capable platform (e.g., Raspberry Pi) or if a rigorous, uncertainty-quantified state estimate becomes a specific research requirement.

8 Conclusion

This report has examined three candidate noise-filtering techniques – the Low-Pass Filter, the Kalman Filter, and the Madgwick Filter – for conditioning the MPU6050 IMU signal of a vibration-based artificial robotic antenna prior to Echo-State-Network-based ego-motion cancellation. The Low-Pass Filter was shown to offer excellent computational efficiency and simplicity but no orientation estimate and no principled way to separate overlapping-frequency ego-motion and contact vibration. The Kalman Filter was shown to offer statistically optimal, uncertainty-aware state estimation and effective sensor fusion, at the cost of higher computational complexity, a larger memory footprint, and a non-trivial tuning and modelling burden that is difficult to justify given the ESN’s role in performing the primary ego-motion separation task. The Madgwick Filter was shown to offer a favourable middle ground: a genuine, drift-corrected orientation estimate obtained through computationally light quaternion and gradient-descent operations, without requiring an explicit dynamic model of the antenna or servo system, and well matched to the resource constraints of the Arduino and ESP32 platforms used in this project.

On the basis of real-time performance, memory usage, computational cost, noise suppression, and suitability for the project’s embedded hardware, the **Madgwick Filter is recommended** as the primary filtering stage to precede the ESN-based ego-motion cancellation network, with an optional lightweight LPF pre-stage for broadband noise removal. The **Kalman Filter remains a valuable alternative** for future work, particularly if the project migrates to more computationally capable hardware or if explicit, uncertainty-quantified state estimates become necessary for more advanced sensor fusion or research contributions.

References

- [1] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, “Estimation of IMU and MARG orientation using a gradient descent algorithm,” in *Proc. IEEE Int. Conf. Rehabilitation Robotics (ICORR)*, Zurich, Switzerland, 2011, pp. 1–7.
- [2] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [3] G. Welch and G. Bishop, “An introduction to the Kalman filter,” Univ. of North Carolina at Chapel Hill, Dept. of Computer Science, Tech. Rep. TR 95-041, 2006.
- [4] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification*, Revision 3.4, 2013.
- [5] A. M. Sabatini, “Quaternion-based extended Kalman filter for determining orientation by inertial and magnetic sensing,” *IEEE Trans. Biomed. Eng.*, vol. 53, no. 7, pp. 1346–1356, Jul. 2006.
- [6] J. Diebel, “Representing attitude: Euler angles, unit quaternions, and rotation vectors,” Stanford Univ., Tech. Rep., 2006.
- [7] S. Haykin, *Adaptive Filter Theory*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2013.
- [8] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [9] H. Jaeger, “The ‘echo state’ approach to analysing and training recurrent neural networks,” German National Research Center for Information Technology, GMD Report 148, 2001.
- [10] T. Hamel and R. Mahony, “Attitude estimation on $SO(3)$ based on direct inertial measurements,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Orlando, FL, USA, 2006, pp. 2170–2175.